Project Walkthrough: Building the Music ML Backend

Hello and welcome! This document explains the complete journey of our machine learning project, from a messy pile of data to two finished, working models.

Our overall goal was to build two key features for a music platform:

1. **A Song Recommender:** "If you like this song, you'll also like these."
2. **A Popularity Predictor:** "Can we predict if a new song will be a hit?"

The project was split into two main phases: **Phase 1: Data Engineering** (Handled by your teammate) **Phase 2: Model Building & Experimentation** (Handled by me)

Here is the story of how we did it, what problems we hit, and how we solved them.

Phase 1: The Foundation (Data Engineering & Cleaning)

You can't build a house on a bad foundation. Our first step was to take a massive, messy dataset and turn it into something a machine learning model could actually use.

The Starting Point: The "Raw" Data

- **What we had:** A single file with 114,000 songs.
- **The Problems:**
    1. **Messy Genres:** There were 114 different genres (e.g., "j-pop," "k-pop," "pop," "indie-pop"). This is too specific and confusing.
    2. **Duplicates:** Thousands of songs were listed multiple times.
    3. **Too Many Features:** There were 11+ columns just for audio features (`danceability` , `energy` , `loudness` , `tempo` , etc.).

The Solution: How We Cleaned It

1. Heavy Cleaning & Deduplication:

- **What:** Your teammate removed all duplicate songs and invalid rows.
- **Why:** If we didn't, our recommender would just recommend the *same song* five times. We needed one unique entry for every track.
- **Result:** This brought our dataset down from 114,000 to **81,343 unique songs**.

2. Feature Engineering: Mapping Genres

- **What:** We manually created a "map" to group the 114 genres into 11 main categories (e.g., "j-pop" and "k-pop" both became "Pop").
- **Why:** This makes the `track_genre` column *useful*. Now we can do real analysis and (as you'll see later) use it as a target for a model.

3. Feature Reduction: The "Smoothie" (PCA)

- **This is the most important "ML" step in the data prep.**
- **The Problem:** The 11+ audio features (`energy` , `loudness` , etc.) are all related. A high-energy song is probably also loud and fast. This "multicollinearity" is bad for models and slow to process.
- **The Solution:** We used **Principal Component Analysis (PCA)**.
- **Simple Analogy:** Think of PCA like making a smoothie. Instead of giving the model 11 separate ingredients (a banana, some milk, ice), you blend them into 5 "super-features" (like "PCA1," "PCA2," etc.). These 5 new features are all that's needed to explain the "taste" of the original 11.
- **Result:** We went from 11+ audio columns to **just 5 "PCA" columns**. This is faster, more efficient, and builds better models.

End of Phase 1: We now have a new, clean file: `processed_data.txt` . It has 81,343 unique songs, 11 clean genres, and 5 powerful PCA features. Now, my work could begin.

Phase 2: My Work (Building the Models)

With the clean data, I was tasked with building the two models. This was a journey of experimenting, failing, and finding better solutions.

Task 1: The Song Recommender (The "Memory Crash")

My goal was to build a "content-based" recommender, which means if you like a song, it recommends other songs that *sound* similar.

The First Attempt: Cosine Similarity (The "Obvious" Way)

- **How it works:** This method creates a giant spreadsheet (a matrix) that calculates a "similarity score" (from 0 to 1) between *every single song* in the dataset.
- **The Problem:** I hit a massive wall. Our dataset has 81,343 songs. To build this matrix, the computer had to calculate $81,343 \times 81,343 = \mathbf{6,616,684,249}$ (6.6 billion) scores.
- **The Result: A MEMORY CRASH.** This calculation required over 50GB of RAM. My code wasn't "buggy"; the *entire method* was wrong for a dataset this big. It just doesn't scale.

The Second Attempt: K-Nearest Neighbors (KNN) (The "Smart" Way)

- **This was the solution.** I had to use a completely different, more advanced method.
- **How it works (Simple Analogy):**
  - Instead of a giant spreadsheet, KNN builds a **"map"** of all 81,343 songs.
  - Each song is a "dot" on this map, and its "coordinates" are its 5 PCA features.
  - When you ask for a recommendation for "Hold On," the model doesn't build the 6.6 billion-item matrix. It just finds the 5 "dots" (neighbors) that are *closest* to "Hold On" on the map.

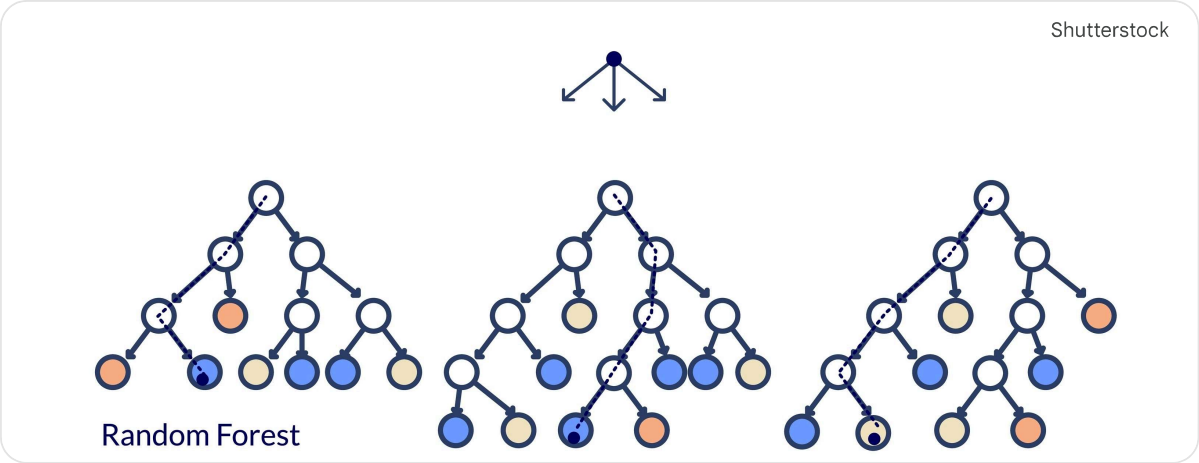
- **The Result:** This is **extremely fast** and uses **almost no RAM**. It's the perfect, scalable solution. This is the final, working recommender in the `recmdr_knn.ipynb` notebook.

Task 2: The Popularity Predictor (The "93% Bug")

My second task was to see if we could predict a song's popularity just from its audio features. This was a *supervised learning* problem.

The First Attempt: The "Buggy" Model

- **How it works:** I used a **Random Forest** model. (Analogy: It's like asking 100 "experts" (trees) to vote on a song's popularity. This is very powerful).



- **The Problem:** My first model gave me **93% accuracy!** This is amazing... but also very suspicious.
- **The Debugging:** I discovered this high score was a "bug" caused by the *old, messy data*. The model wasn't learning the *sound*; it was just memorizing that "j-pop" (one of the 114 genres) was popular. This is a classic "data leakage" problem.

The Second Attempt: The Final, "Honest" Model

- **The Solution:** I re-trained my Random Forest using the **new, clean data** (with only 11 genres).
- **The Result:** The new, *honest* accuracy is **61.88%**.
- **Why this is a GOOD result:** The 93% was a lie. This 61.88% is *real*. It tells us the *true* power of a song's audio features. We now have a real, working model that we can improve on.
- **Final Step:** I saved this final, trained model as `popularity_model.pkl`. This "pickled" file is the "brain" of the model, ready to be put into a live app without needing to be retrained.

Final Summary

So, at the end of this whole process, our team has:

1. **A Clean Dataset:** `processed_data.txt` is our "single source of truth."
2. **A Scalable Recommender:** A working KNN model that can handle millions of songs without crashing.

3. **An Honest Predictor:** A Random Forest model that gives a *real* 62% accuracy on popularity, which we've saved and can use in production.

The biggest lessons were that data cleaning is 90% of the job, and the "simple" solution (Cosine Similarity) often breaks, forcing you to find a more robust, scalable one (KNN).